23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : B10

Roll No	Student Name
AM.SC.U4CSE25119	GOUTHAM
AM.SC.U4CSE25159	YADHU KRISHNA LIJU
AM.SC.U4CSE25167	MYDHILY MS
AM.SC.U4CSE25168	MEERA KRISHNA S

Project Title : HOSPITAL MANAGEMENT SYSTEM

CONTENTS

SI NO	Chapter	Page No
1	Abstract	3
2	Introduction	3
3	Requirements (Functional & Non-functional)	4
4	UI Design	5 - 8
5	Database Design	8 - 17
6	Database Selection & Justification	17 - 18
7	Implementation	18 - 22
8	Testing & Validation	22 - 23
9	Results & Screenshots	23 - 26
10	Challenges faced & solutions	27
11	Conclusion & future enhancements	27
12	References	28

Abstract

The Hospital Management System (HMS) is a database-driven application developed to address the difficulties of manually managing patient records, appointments, prescriptions, and billing, which can be time-consuming and prone to errors.The main objective is to simplify hospital operations by providing a centralized system for storing, organizing, and retrieving information efficiently. The proposed system supports patient registration, doctor management, appointment scheduling, prescription management, billing, and report generation. It incorporates key DBMS features such as entity-relationship modeling, normalization, primary and foreign keys, integrity constraints, and SQL operations to maintain data consistency and accuracy. The system enables efficient searching, updating, and retrieval of records while improving data organization and security. The expected outcome is a reliable and user-friendly solution that reduces paperwork, minimizes errors, improves hospital administration, and supports better healthcare service management.

2. Introduction

2.1 Background

A hospital is a data-intensive organisation in which several departments work with overlapping information. Registration staff maintain patient details, doctors review appointments and issue prescriptions, pharmacy staff manage medicine stock, laboratories record results, and administrative teams manage rooms, bills and staff records. These activities need a shared and reliable source of information.

2.2 Existing scenario and limitations

When information is stored in paper registers, spreadsheets or disconnected applications, the same details may be entered more than once. Records can become inconsistent; retrieving a patient's history may require searching several sources, and simultaneous changes may conflict. Manual scheduling can also lead to overlapping appointments or room and theatre bookings. Inventory records can become inaccurate if dispensing is not reflected promptly.

2.3 Need for the proposed system

A centralized relational database provides structured storage and links related records through keys. Constraints can reject invalid values, foreign keys maintain referential integrity, and queries can combine information from multiple tables. Database functions and triggers can enforce selected business rules close to the stored data. A web interface and API make these records available to authorised application users.

2.4 Proposed system and objectives

MediCare is a multi-branch Hospital Management System using PostgreSQL, a Node.js/Express REST API and a browser-based interface. Its objectives are to:

- organise hospital information in related relational tables;
- support patient, doctor, appointment, admission, prescription, pharmacy, laboratory, theatre, billing and staff workflows;
- maintain data integrity through primary keys, foreign keys, unique constraints, check constraints and database logic;
- provide search, create, update and delete operations where supported by the application;
- present useful dashboard and report information; and
- demonstrate DBMS concepts through schema design, SQL queries, constraints, indexes, functions, triggers and views.

3. Requirements (Functional & Non-functional)

3.1 Functional requirements

Module	Required operation
Authentication	Allow a user to sign in and apply role-based access through the API.
Patients	Create, view, search and update patient records; support deletion where permitted.
Doctors and departments	Maintain doctor information and associated department/branch data.
Appointments	Create and manage appointments, including date/time and status.
Admissions and rooms	Maintain room information and admission/discharge records.
Prescriptions	Record prescriptions and their medicine details.
Pharmacy and medicines	Maintain medicine information and pharmacy-specific stock.
Laboratory	Store patient-related laboratory results.
Operation theatre	Maintain theatre and procedure schedule records.
Billing	Maintain bill information and payment status/mode.
Staff and payroll	Maintain staff, shift logs, pay structures and salary records.
Dashboard and reports	Display summary information and query-based reports.

3.2 Non-functional requirements

Quality	Requirement
Data integrity	Use keys, constraints and database logic to reduce invalid or inconsistent records.
Security	Protect routes using authentication/authorization mechanisms configured in the backend; keep secrets in environment configuration.
Reliability	Handle database and application errors without reporting a successful operation when it fails.
Performance	Use indexes for selected query patterns and reuse database connections through the configured client/pool.
Usability	Provide a browser-based interface with navigation to the available modules.
Maintainability	Separate SQL scripts, routes, controllers, middleware and frontend assets.
Scalability	Use a relational schema and API structure that can be extended with additional records and modules.

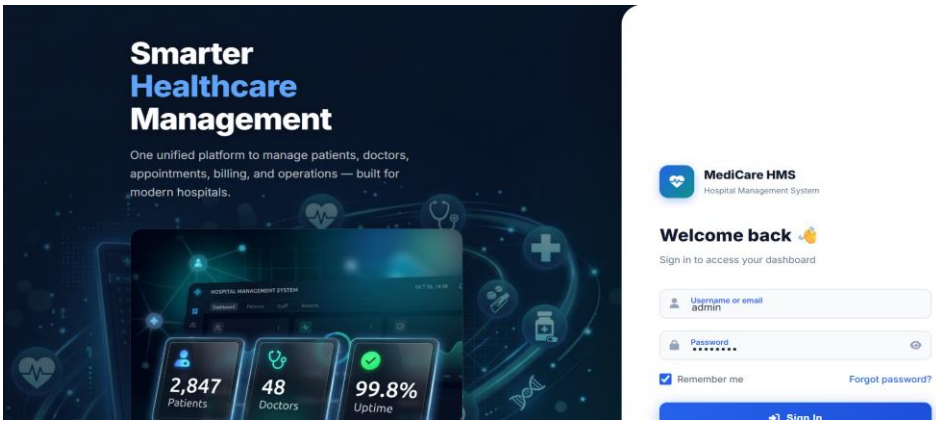
4. UI Design

The MediCare Hospital Management System provides a user-friendly interface for managing hospital operations. The interface is organized into separate modules so that users can access patient records, doctor information, appointments, prescriptions, pharmacy inventory, billing, admissions, staff details, and reports.

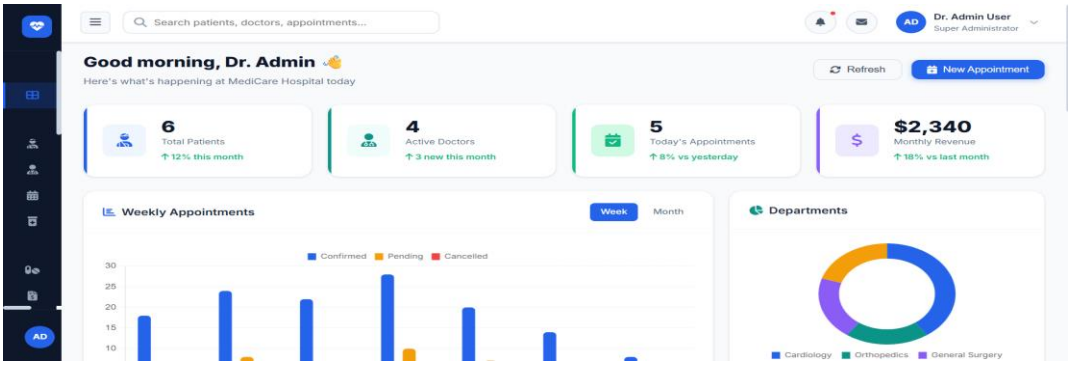
4.1 Navigation flow

```
Open login page
      ↓
Enter credentials
      ↓
Authentication response
      ↓
Dashboard
      ↓
Patients / Doctors / Appointments /
Pharmacy / Prescriptions       ↓
Billing / Staff / Laboratory / Operation Theatre
/ Reports / Settings       ↓
Perform an available operation or
logout
```

4.2. Login Screen

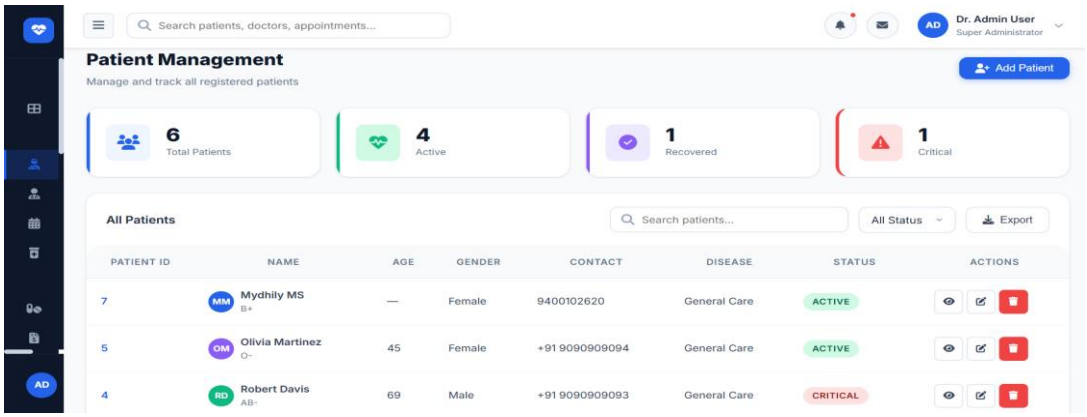


4.3. Dashboard

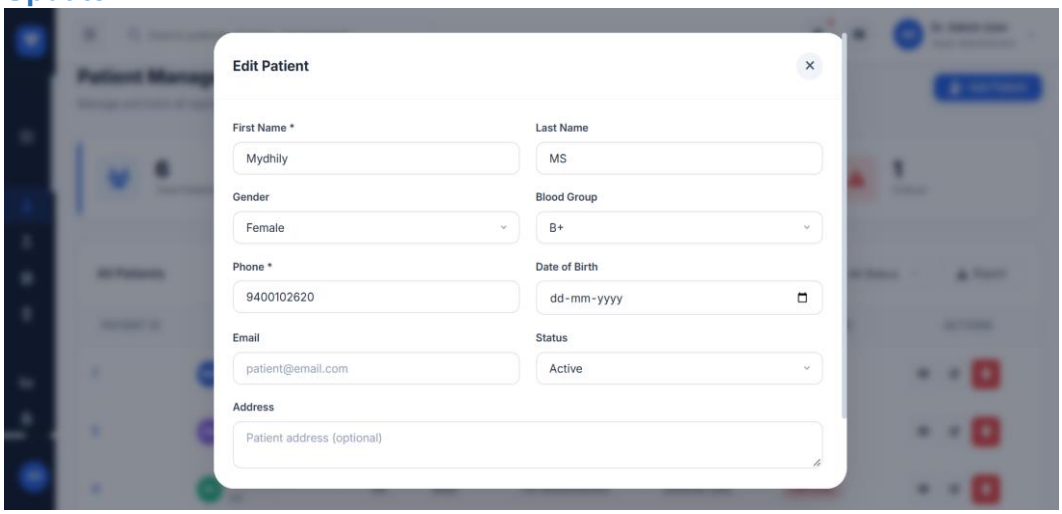


- The dashboard displays key statistics such as total patients, active doctors, appointments, revenue, weekly appointment statistics, and patient statistics.

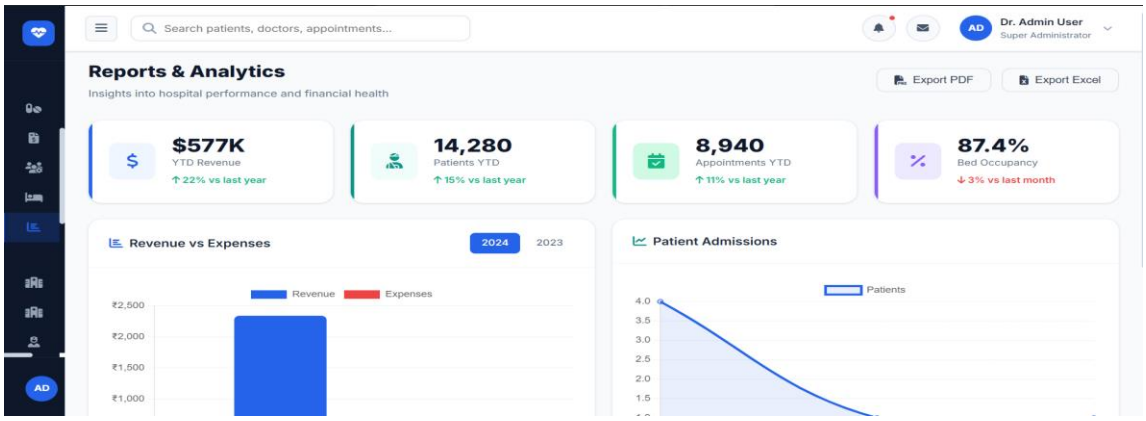
4.4 Data Entry



Update:



Report generation :



4.2 Important screens

Screen	Purpose
Login	Accepts user credentials and initiates authentication.
Dashboard	Provides an entry point to modules and displays summary information supported by the application.
Patients	Provides patient listing and patient-record operations.
Doctors	Displays doctor information and related operations.
Appointments	Provides appointment listing and booking/management controls.
Pharmacy and prescriptions	Supports medicine/inventory and prescription-related workflows.
Billing	Displays billing records and associated operations.
Staff	Provides staff-related records and controls.

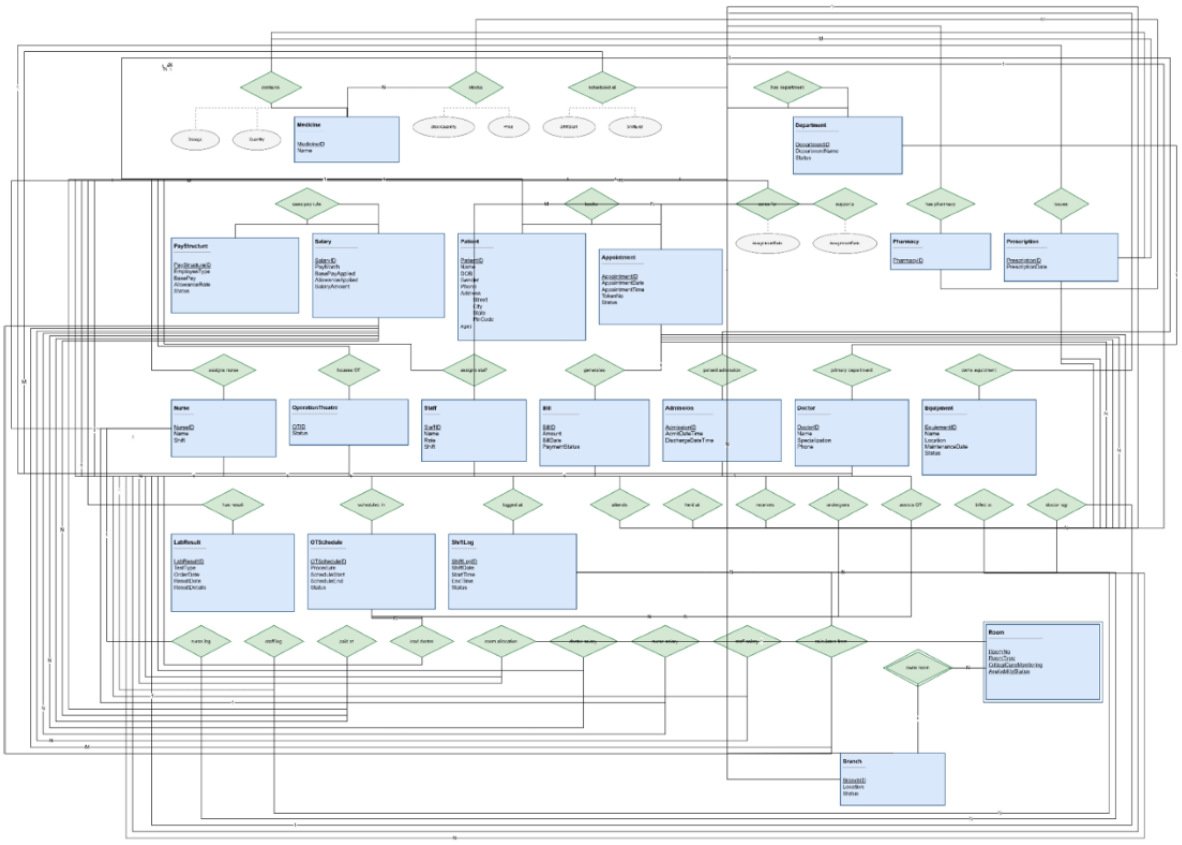
Reports	Presents reporting information available through the backend.
Settings	Contains application settings controls available in the supplied frontend.

4.3 User interaction

Users interact with the MediCare Hospital Management System through a user-friendly web interface. After logging in, users can access the dashboard to view an overview of hospital activities. They can navigate through different modules to manage patient records, doctor details, appointments, prescriptions, pharmacy inventory, billing, admissions, staff, and other hospital operations. Users can enter new information using data entry forms, search for existing records using search and filter options, and modify records through the available update functions. The Reports and Analytics module allows users to view summarized hospital information through statistics and charts, with options to export reports where supported. The system uses a PostgreSQL database to store and retrieve information, enabling users to manage hospital data through the interface instead of directly executing SQL queries.

5. Database Design

5.1 ER DIAGRAM



5.2 Entities, attributes and keys

Entity/table	Main purpose	Primary key / important links
branch	Hospital branch details	branch_id

department	Department information	department_id; branch_id FK
doctor	Doctor details and speciality	doctor_id; branch_id, department_id FKs
doctor_shift	Doctor duty periods at branches	shift_id; doctor_id, branch_id FKs
patient	Patient demographic/contact data	patient_id
appointment	Patient-doctor appointment	appointment_id; patient_id, doctor_id, department_id, branch_id FKs
nurse	Nurse details	nurse_id; branch_id, department_id FKs
staff	Support/administrative staff	staff_id; branch_id, department_id FKs
room	Rooms associated with a branch	room_id; branch_id FK; unique branch_id + room_number
admission	Patient admission and room allocation	admission_id; patient_id, doctor_id, room_id, branch_id, department_id FKs
medicine	Medicine catalogue	medicine_id
pharmacy	Pharmacy at a branch	pharmacy_id; branch_id FK
pharmacy_inventory	Stock for a medicine at a pharmacy	inventory_id; pharmacy_id, medicine_id FKs; pair is unique
prescription	Prescription header	prescription_id; patient_id, doctor_id, branch_id FKs
prescription_detail	Medicine items on a prescription	detail_id; prescription_id, medicine_id FKs; pair is unique
lab_result	Patient laboratory result	lab_result_id; patient_id, doctor_id FKs
operation_theatre	Theatre details	theatre_id; branch_id FK
ot_schedule	Scheduled procedure	schedule_id; theatre_id, doctor_id, patient_id FKs
equipment	Equipment assigned to a theatre	equipment_id; theatre_id FK
bill	Patient bill and payment status	bill_id; patient_id, doctor_id, appointment_id, admission_id, branch_id FKs
shift_log	Staff shift record	shift_log_id; staff_id, branch_id FKs
pay_structure	Pay configuration for staff	pay_structure_id; staff_id FK
salary	Salary record for a period	salary_id; staff_id FK

5.3 Main relationships and cardinalities

Relationship	Cardinality	Explanation
Branch–Department	1:N	A branch can have several departments; each department references a branch.
Department–Doctor	1:N	A department can be associated with multiple doctors.
Doctor–DoctorShift	1:N	A doctor can have multiple shift records.

Branch–DoctorShift	1:N	A branch can have multiple doctor shift records.
Patient–Appointment	1:N	A patient can have multiple appointments.
Doctor–Appointment	1:N	A doctor can attend multiple appointments.
Branch–Room	1:N	A branch can contain multiple rooms; room numbers are unique within a branch.
Patient–Admission	1:N	A patient can have admission records over time.
Room–Admission	1:N over time	A room can appear in multiple admission records at different times.
Prescription–PrescriptionDetail	1:N	A prescription can contain multiple medicine-detail rows.
Medicine–PrescriptionDetail	1:N	A medicine can occur in details for many prescriptions.
Prescription–Medicine	M:N resolved by PrescriptionDetail	The junction table stores dosage, frequency, duration and quantity details.
Pharmacy–PharmacyInventory	1:N	A pharmacy has inventory records for medicines.
Medicine–PharmacyInventory	1:N	A medicine can be stocked by different pharmacies.
Pharmacy–Medicine	M:N resolved by PharmacyInventory	The junction table stores pharmacy-specific stock.
OperationTheatre–OTSchedule	1:N	A theatre can have multiple scheduled procedures at different times.
Staff–ShiftLog	1:N	A staff member can have multiple shift logs.
Staff–Salary	1:N	A staff member can have salary records for different periods.

5.4 Relational schema

1. Hospital and department management

BRANCH (branch_id PK,
  branch_name, address, phone, email, status,
  created_at, updated_at)
DEPARTMENT (
  department_id PK,
  branch_id FK → BRANCH (branch_id),
  department_name, description, status,
  created_at, updated_at,
  UNIQUE(branch_id, department_name))

DOCTOR (
  doctor_id PK,
  branch_id FK → BRANCH(branch_id),

department_id FK → DEPARTMENT(department_id),
doctor_name, specialization, license_no, phone, email,
status, consultation_fee, created_at, updated_at,
UNIQUE (license_no),
UNIQUE (email)))

DOCTOR_SHIFT (
shift_id PK,
doctor_id FK → DOCTOR(doctor_id),
branch_id FK → BRANCH(branch_id),
shift_date, start_time, end_time, status,
created_at, updated_at
)

2. Patient and appointment management

PATIENT (
patient_id PK,
first_name, last_name, date_of_birth, gender, blood_group,
phone, email, address, status, created_at, updated_at
)

APPOINTMENT(
appointment_id PK,
patient_id FK → PATIENT(patient_id),
doctor_id FK → DOCTOR(doctor_id),
department_id FK → DEPARTMENT(department_id),
branch_id FK → BRANCH(branch_id),
appointment_date, start_time, end_time,
appointment_type, status, notes, created_at, updated_at)

NURSE(
nurse_id PK,
branch_id FK → BRANCH(branch_id),
department_id FK → DEPARTMENT(department_id),
nurse_name, phone, email, status, created_at, updated_at, UNIQUE(email))

STAFF(

staff_id PK,
branch_id FK → BRANCH(branch_id),
department_id FK → DEPARTMENT(department_id),
staff_name, role, phone, email, hire_date, status,
created_at, updated_at, UNIQUE(email))

3. Room and admission management

ROOM(
room_id PK,
branch_id FK → BRANCH(branch_id),
room_number, room_type, capacity, status,
created_at, updated_at,
UNIQUE(branch_id, room_number)
)

ADMISSION(
admission_id PK,
patient_id FK → PATIENT(patient_id),
doctor_id FK → DOCTOR(doctor_id),
room_id FK → ROOM(room_id),
branch_id FK → BRANCH(branch_id),
department_id FK → DEPARTMENT(department_id),
admitted_on, discharged_on, status, notes,
created_at, updated_at
)

4. Medicine and pharmacy management

MEDICINE(
medicine_id PK,
medicine_name UNIQUE, generic_name, category, unit_price,
stock_quantity, reorder_level, expiry_date, supplier, status,
created_at, updated_at
)

PHARMACY(
pharmacy_id PK,
branch_id FK → BRANCH(branch_id),

pharmacy_name, phone, status, created_at, updated_at,

UNIQUE(branch_id, pharmacy_name)

)

PHARMACY_INVENTORY(

inventory_id PK,

pharmacy_id FK → PHARMACY(pharmacy_id),

medicine_id FK → MEDICINE(medicine_id),

quantity, reorder_level, last_updated,

UNIQUE(pharmacy_id, medicine_id)

)

5. Prescription and laboratory management

PRESCRIPTION(

prescription_id PK,

patient_id FK → PATIENT(patient_id),

doctor_id FK → DOCTOR(doctor_id),

branch_id FK → BRANCH(branch_id),

prescription_date, diagnosis, notes, status,

created_at, updated_at

)

PRESCRIPTION_DETAIL(

detail_id PK,

prescription_id FK → PRESCRIPTION(prescription_id),

medicine_id FK → MEDICINE(medicine_id),

dosage, frequency, duration_days, quantity, instructions, UNIQUE(prescription_id, medicine_id))

LAB_RESULT(

lab_result_id PK,

patient_id FK → PATIENT(patient_id),

doctor_id FK → DOCTOR(doctor_id),

test_name, result_value, normal_range, status,

report_date, created_at, updated_at

)

6. Operation theatre and equipment management

OPERATION_THEATRE(
  theatre_id PK,
  branch_id FK → BRANCH(branch_id),
  theatre_name, status, created_at, updated_at,
  UNIQUE(branch_id, theatre_name)
)

OT_SCHEDULE(
  schedule_id PK,
  theatre_id FK → OPERATION_THEATRE(theatre_id),
  doctor_id FK → DOCTOR(doctor_id),
  patient_id FK → PATIENT(patient_id),
  procedure_name, scheduled_date, start_time, end_time,
  status, created_at, updated_at
)

EQUIPMENT(
  equipment_id PK,
  theatre_id FK → OPERATION_THEATRE(theatre_id),
  equipment_name, quantity, status, last_service_date,
  created_at, updated_at
)

7. Billing and staff payroll

BILL(
  bill_id PK,
  patient_id FK → PATIENT(patient_id),
  doctor_id FK → DOCTOR(doctor_id),
  appointment_id FK → APPOINTMENT(appointment_id),
  admission_id FK → ADMISSION(admission_id),
  branch_id FK → BRANCH(branch_id),
  bill_date, total_amount, status, payment_mode, notes,
  created_at, updated_at
)

SHIFT_LOG(
  shift_log_id PK,

staff_id FK → STAFF(staff_id),

branch_id FK → BRANCH(branch_id),

shift_date, start_time, end_time, status,

created_at, updated_at

)

PAY_STRUCTURE(

pay_structure_id PK,

staff_id FK → STAFF(staff_id) UNIQUE,

position_title, base_salary, allowance, overtime_rate,

effective_from, created_at, updated_at

)

SALARY(

salary_id PK,

staff_id FK → STAFF(staff_id),

pay_period_start, pay_period_end, gross_salary,

deductions, net_salary, status, created_at, updated_at

)

8. User authentication

USER_ACCOUNT (

user_id PK,

username UNIQUE, password_hash, role,

staff_id FK → STAFF(staff_id),

doctor_id FK → DOCTOR(doctor_id),

patient_id FK → PATIENT(patient_id),

status, created_at, updated_at

)

5.5 Constraints

• **Primary keys** uniquely identify rows in each table.
• **Foreign keys** link dependent records to their parent records.
• **Unique** constraints protect values such as usernames, doctor licence identifiers and selected table pairs.

- **Check** constraints validate selected statuses, quantities, amounts and time/date relationships where defined in the SQLscripts.
- **Triggers and functions** implement additional business rules such as scheduling and inventory validation where defined in the database scripts.

5.6 Functional dependencies

Functional dependency	Meaning
branch_id → branch_name, address, phone, email, status	The branch identifier determines the branch record.
department_id → branch_id, department_name, description, status	The department identifier determines the department record, including its branch reference.
doctor_id → doctor_name, specialization, license_no, phone, email, consultation_fee, status	The doctor identifier determines the doctor record.
patient_id → first_name, last_name, date_of_birth, gender, blood_group, phone, email, address, status	The patient identifier determines the patient record.
appointment_id → patient_id, doctor_id, department_id, branch_id, appointment_date, start_time, end_time, status	The appointment identifier determines one appointment record.
prescription_id → patient_id, doctor_id, branch_id, prescription_date, diagnosis, notes, status	A prescription ID identifies one prescription.
(prescription_id, medicine_id) → prescription-detail attributes	The prescription/medicine pair identifies the associated medicine-detail entry where the pair is constrained unique.
(pharmacy_id, medicine_id) → inventory attributes	The pharmacy/medicine pair identifies the pharmacy-specific inventory entry where constrained unique.
(branch_id, room_number) → room attributes	The branch and room number combination identifies a room under the unique constraint.

5.1 Conversion of the ER Model into Relational Tables

- **Strong entities:** Each strong entity is converted into a table with a primary key.
- **One-to-many (1:N) relationships:** The primary key of the one-side entity is added as a foreign key to the many-side table.
- **Many-to-many (M:N) relationships:** A separate junction table is created containing foreign keys referencing the related tables.
- **Composite attributes:** Composite attributes are divided into their individual components where applicable.

- **Key constraints:** Primary keys, foreign keys, and unique constraints maintain record identification and relationship integrity.

5.7 Normalization

Normalization organizes relations to minimize unnecessary redundancy and avoid insertion, update, and deletion anomalies.

First Normal Form (1NF): Each table stores attributes as individual values rather than repeating groups. Multiple medicines in a prescription are stored as separate records in `prescription_detail`.

Second Normal Form (2NF): Non-key attributes should depend on the whole candidate key, not merely part of a composite candidate key. For example, medicine descriptions are stored in `medicine` instead of being repeated in pharmacy inventory records.

Third Normal Form (3NF): Non-key attributes should not depend transitively on a candidate key. For example, department descriptions are stored in `department` rather than being repeated in every doctor record.

Boyce–Codd Normal Form (BCNF): For each non-trivial functional dependency, the determinant must be a superkey. BCNF should be claimed only after verifying the dependencies and candidate keys of the actual implemented relations.

6. Database Selection & Justification

- The MediCare Hospital Management System uses PostgreSQL as its database management system.

1. Relational data management: It supports related tables, primary keys, foreign keys, and constraints, which are essential for connecting patients, doctors, appointments, admissions, billing, and other hospital records.

2. Data integrity: PostgreSQL supports constraints and transactions to help maintain accurate and consistent hospital information.

3. Advanced SQL features: It supports SQL queries, joins, aggregate functions, views, indexes, stored functions, and triggers, which are used in MediCare for data retrieval, reporting, validation, and automation.

4. Performance: Indexes help improve the speed of frequently used queries, such as searching for appointments or retrieving patient records.

5. Security: PostgreSQL provides user and role management, privileges, and authentication mechanisms to help protect database access.

6. Scalability : PostgreSQL supports transactions and recovery mechanisms and can handle growing amounts of structured data as the application expands.

7. Cost-effectiveness: As an open-source DBMS, PostgreSQL does not require a proprietary database licence, making it suitable for an academic project.

8. Integration: PostgreSQL integrates with the Node.js and Express backend used in MediCare, enabling the application to communicate with the database and perform database operations.

.

7. Implementation

The Hospital Management System is implemented using a three-tier architecture. The frontend is developed using HTML, CSS, and JavaScript. The backend uses Node.js and Express to handle API requests, authentication, and communication with the database. PostgreSQL stores the hospital records and maintains relationships between the tables.

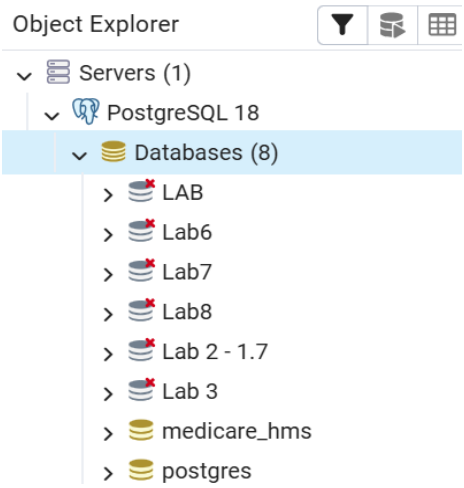
The database scripts are organized into separate files for schema creation, constraints, indexes, functions, triggers, views, sample data, and test queries. This separation makes the database easier to set up, maintain, and test.

7.1 Technology stack

Layer	project component
Frontend	HTML, CSS and JavaScript in the DBMS FRONTEND directory.
Backend	Node.js and Express routes, controllers and middleware.
Database	PostgreSQL.
Database scripts	01_schema.sql, 02_constraints.sql, 03_indexes.sql, 04_functions.sql, 05_triggers.sql, 06_views.sql, 07_seed.sql and 08_test_queries.sql.
Backend tests	A test file is present at backend/tests/hms.test.js; execute it to capture the actual result.

7.2 Database Creation

The Medicare Hospital Management System uses PostgreSQL as its database management system. A database named `medicare_hms` is used to store and manage hospital information. It acts as the central storage system for the application's data.



7.3 Table Creation

The database consists of 24 relational tables designed to organize the information required for hospital management. These tables store information about branches, departments, doctors, patients, appointments, admissions, prescriptions, billing, and other hospital operations. Primary keys are used to identify records, while foreign keys establish relationships between related tables.

Appointment Table :

```
CREATE TABLE IF NOT EXISTS appointment (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE RESTRICT,
    doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
    department_id INT NOT NULL REFERENCES department(department_id) ON DELETE RESTRICT,
    branch_id INT NOT NULL REFERENCES branch(branch_id) ON DELETE RESTRICT,
    appointment_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    appointment_type VARCHAR(50) NOT NULL DEFAULT 'consultation' CHECK (appointment_type IN ('consultation','follow_up','emergency'
    ,'checkup')),
    status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending','confirmed','completed','cancelled')),
    notes TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CHECK (end_time > start_time)
);
```

Department Table :

```
CREATE TABLE IF NOT EXISTS department (
    department_id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES branch(branch_id) ON DELETE RESTRICT,
    department_name VARCHAR(100) NOT NULL,
    description TEXT,
    status VARCHAR(20) NOT NULL DEFAULT 'active' CHECK (status IN ('active','inactive')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (branch_id, department_name)
);
```

7.3 Constraints

Constraints are used in the Medicare Hospital Management System database to maintain data integrity and consistency. Primary keys uniquely identify each record, foreign keys establish relationships between related tables, UNIQUE constraints prevent duplicate values, NOT NULL constraints ensure that required fields contain values, and CHECK constraints restrict data to permitted values. These constraints help prevent invalid data from being stored in the database.

- Primary Key: Identifies each department uniquely.

```
department_id SERIAL PRIMARY KEY,
```

- Foreign Key: Links a department to its corresponding branch.

```
branch_id INT NOT NULL REFERENCES branch(branch_id) ON DELETE RESTRICT,
```

- UNIQUE: Prevents duplicate department names within the same branch.

```
UNIQUE (branch_id, department_name)
```

- CHECK: Allows only the permitted department status values.

```
CHECK (status IN ('active','inactive')),
```

7.4 Sample Data:

```
INSERT INTO department (branch_id, department_name, description, status)
VALUES
    (1, 'Cardiology', 'Heart care and monitoring', 'active'),
    (1, 'Neurology', 'Brain and nervous system care', 'active'),
    (1, 'General Surgery', 'Surgical treatments and procedures', 'active'),
    (2, 'Orthopedics', 'Bone and joint care', 'active'),
    (2, 'Pediatrics', 'Child health services', 'active'),
    (3, 'Dermatology', 'Skin and dermatology care', 'inactive');
```

	department_id [PK] integer	branch_id integer	department_name character varying (100)	description text	status character varying (20)
1	1	1	Cardiology	Heart care and monitoring	active
2	2	1	Neurology	Brain and nervous system care	active
3	3	1	General Surgery	Surgical treatments and procedur...	active
4	4	2	Orthopedics	Bone and joint care	active
5	5	2	Pediatrics	Child health services	active
6	6	3	Dermatology	Skin and dermatology care	inactive

7.4 SQL queries

1. JOIN

```
12    SELECT *
13    FROM patient p
14    JOIN appointment a
15    ON a.patient_id = p.patient_id
16    LIMIT 10;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	patient_id integer	first_name character varying (80)	last_name character varying (80)	date_of_birth date	gender character varying (20)	blood_group character varying (5)	phone character varying (30)
1	1	Emma	Johnson	1990-04-15	Female	A+	+91 9090909090
2	2	Michael	Chen	1972-08-20	Male	O+	+91 9090909091
3	3	Sophia	Williams	1994-01-05	Female	B+	+91 9090909092
4	4	Robert	Davis	1957-09-16	Male	AB-	+91 9090909093
5	5	Olivia	Martinez	1981-11-30	Female	O-	+91 9090909094

2.SELECT

```
12    SELECT *
13    FROM doctor
14    WHERE doctor_id IN (
15    SELECT doctor_id
16    FROM appointment
17    WHERE status = 'confirmed'
18    );
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	nent_id	doctor_name character varying (150)	specialization character varying (100)	license_no character varying (50)	phone character varying (30)	email character varying (100)	status character varying (20)
1	1	Dr. Sarah Chen	Cardiology	LIC-1001	+91 9876543210	sarah.chen@medicare.com	available
2	4	Dr. Ahmed Hassan	Orthopedics	LIC-1004	+91 9876543213	ahmed.hassan@medicare.c...	available
3	3	Dr. Priya Nair	General Surgery	LIC-1003	+91 9876543212	priya.nair@medicare.com	available

3. Aggregation & GROUP BY

```
12    SELECT department_id, COUNT(*) AS appointment_count
13    FROM appointment
14    GROUP BY department_id
15    HAVING COUNT(*) > 0;
```

Data Output | Messages | Notifications

Showing rows:

	department_id integer	appointment_count bigint
1	3	1
2	4	1
3	2	1
4	1	2

4.SUBQUERY :

```
SELECT *
FROM doctor
WHERE doctor_id IN (
SELECT doctor_id
FROM appointment
WHERE status = 'confirmed'
);
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	doctor_id [PK] integer	branch_id integer	department_id integer	doctor_name character varying (150)	specialization character varying (100)	license_no character varying (50)	phone character varying (30)	email character varying
1	1	1	1	Dr. Sarah Chen	Cardiology	LIC-1001	+91 9876543210	sarah.chen@medi
2	4	2	4	Dr. Ahmed Hassan	Orthopedics	LIC-1004	+91 9876543213	ahmed.hassan@m
3	3	1	3	Dr. Priya Nair	General Surgery	LIC-1003	+91 9876543212	priya.nair@medica

7.4 CRUD operations

Operation	Meaning	Example
Create	Insert a new row.	Register a patient or create an appointment.
Read	Retrieve existing rows.	Search patients or list appointments.
Update	Modify an existing row.	Update a patient record or appointment status.
Delete	Remove a row when allowed by integrity rules.	Delete a record only when permitted by foreign-key rules and application logic.

8. Testing & Validation

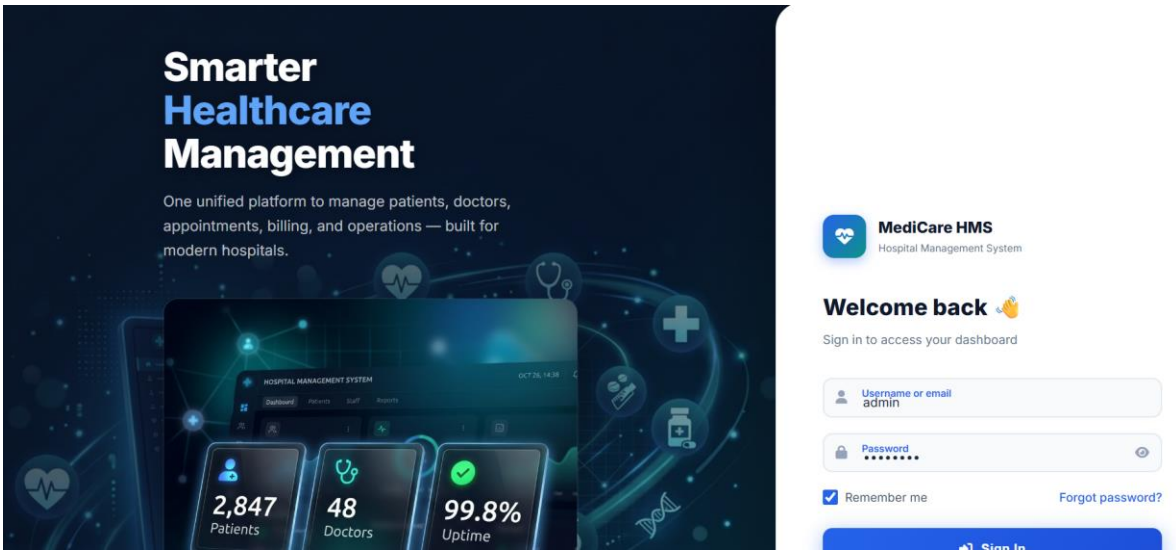
Test	Input/action	Expected result
1	Connect to the configured PostgreSQL database.	Connection succeeds.
2	Query the list of tables.	Expected schema tables are listed.
3	Insert a valid patient record.	Record is inserted and can be retrieved.
4	Query appointments joined with patient and doctor.	Matching linked details are returned.
5	Insert a record with an invalid foreign key.	Constraint rejects the invalid reference.
6	Insert an invalid quantity/amount where a check applies.	Constraint rejects the invalid value.
7	Attempt an overlapping appointment if the configured rule covers it.	The configured validation rejects the conflict.

8	Attempt a stock operation exceeding available inventory if implemented.	The configured logic rejects the invalid operation.
9	Submit invalid login credentials.	Authentication fails without issuing a valid session/token.
10	Call a protected endpoint without valid authorization.	The API rejects the request.

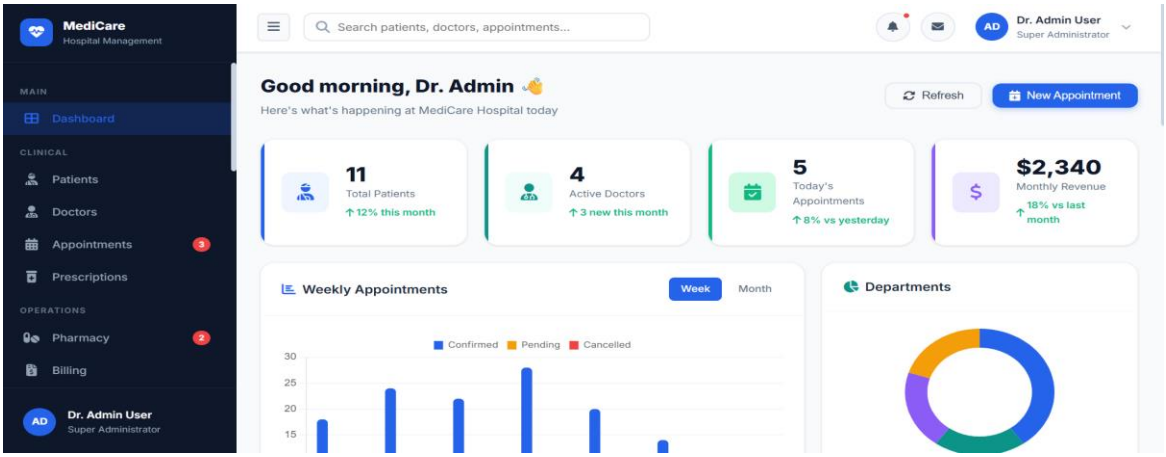
9. Results & Screenshots

Figure	Evidence to capture	What it should demonstrate
9.1	Login screen	Login interface and authentication flow.
9.2	Dashboard	Dashboard cards or reports visible in the running app.
9.3	Patient page	Patient list/search and a valid patient record.
9.4	Appointment page	Appointment list or successful booking.
9.5	Database tables/query output	Tables created in PostgreSQL and a SELECT result.
9.6	Joined SQL query output	Data retrieved by joining related tables.
9.7	Constraint validation output	An invalid operation rejected by a constraint, with the actual error.
9.8	Pharmacy/prescription page	Relevant inventory or prescription workflow.
9.9	Billing or reports page	A real billing/report view from the application.
9.10	Automated test output	Actual test runner summary.

1. Login Screen

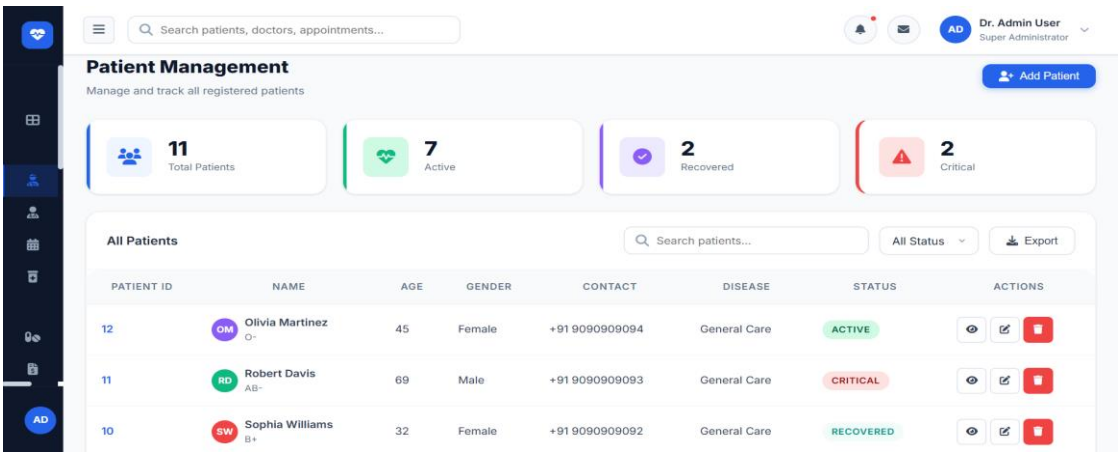


2. Dashboard :

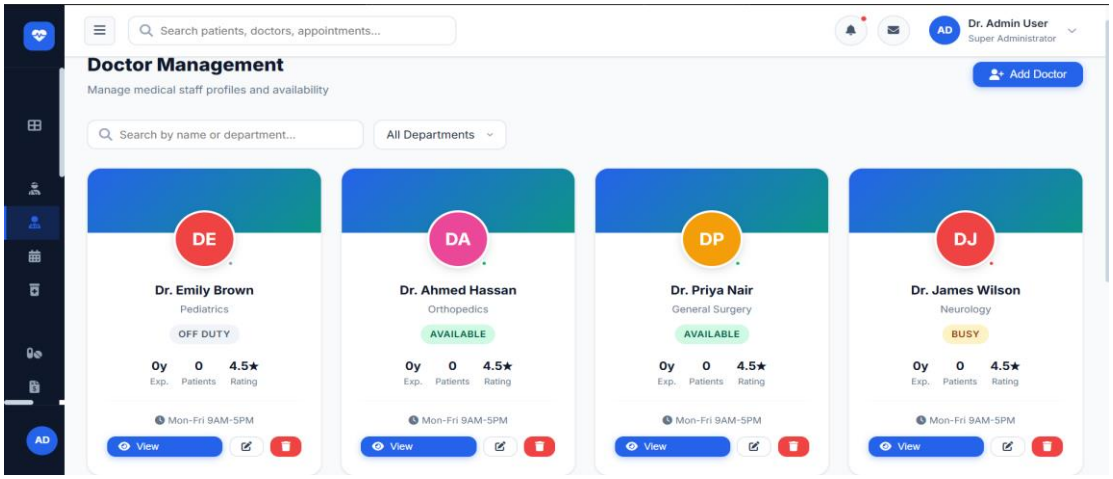


- This screenshot shows the main dashboard of the system, displaying hospital statistics and visual summaries of appointments, patients, revenue, and department-related information.

3.Patient & Doctor Management

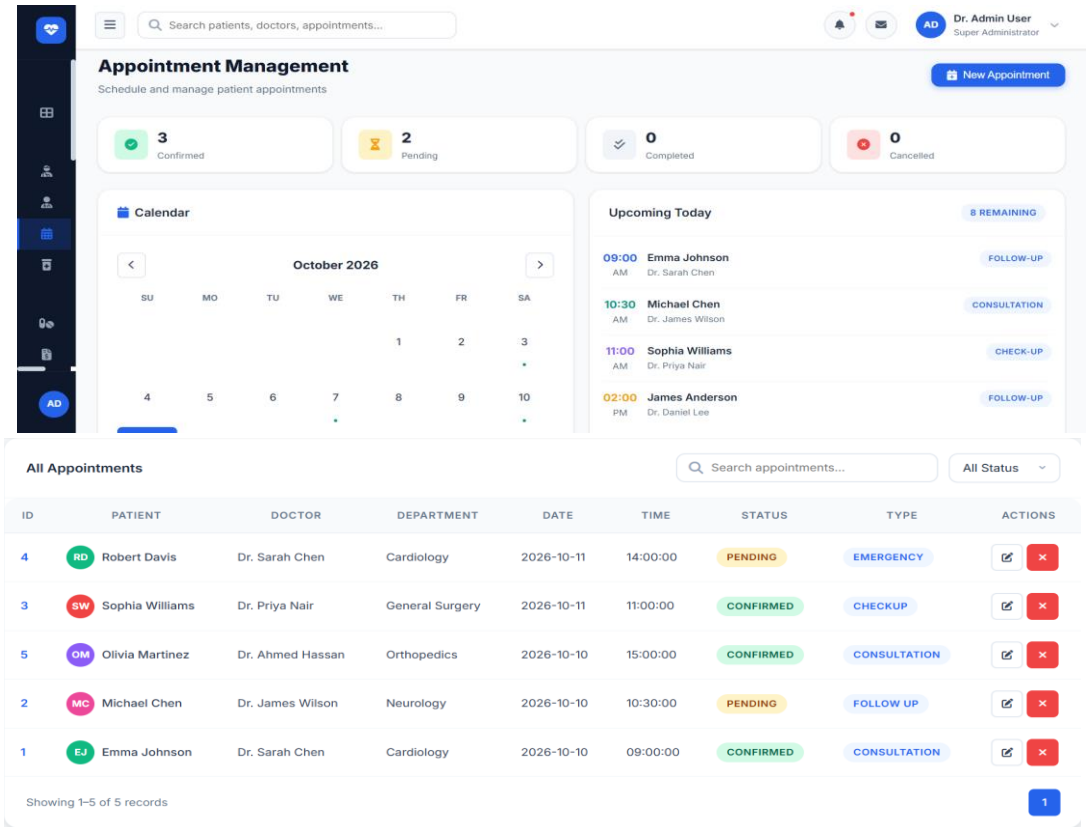


- This screenshot demonstrates the interface used to view and manage patient records

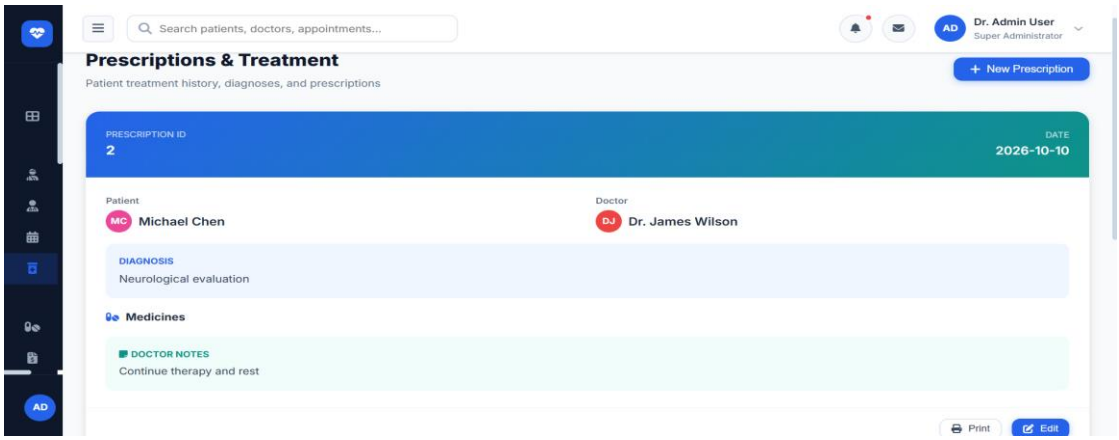


- This screenshot demonstrates how doctor information and related details are displayed and managed in the system.

4. Appointment and Prescription Management



- This shows the appointment management interface, where appointment details and their statuses can be viewed.



- This demonstrates the interface for viewing prescription details and prescribed medicines.

10. Challenges Faced & Solutions

Challenge	Possible solution to report if it matches the team's actual work
Mapping many-to-many relationships	Use junction tables such as prescription_detail and pharmacy_inventory, with foreign keys and uniqueness constraints.
Keeping linked records consistent	Use primary/foreign keys and suitable referential actions.
Handling invalid input	Use application validation and database constraints; display meaningful errors.
Database connectivity and configuration	Verify PostgreSQL service status, database credentials, environment variables and the configured API port.
Managing schema scripts	Run the scripts in the documented order and inspect errors after each script.
Testing business rules	Create valid and invalid test cases and compare actual results with expected results.
Differences between design and implementation	Use the implemented SQL schema as the final source of truth and document important differences from the earlier design model.

11. Conclusion & Future Enhancements

11.1 Conclusion

11.1 Conclusion

The MediCare Hospital Management System was successfully developed as a web-based application integrating a frontend interface, Node.js and Express backend, and PostgreSQL database. The system provides centralized management of essential hospital operations, including patient records, doctor information, appointments, prescriptions, pharmacy inventory, billing, and admissions. By implementing relational database design, integrity constraints, SQL queries, indexes, functions, triggers, and views, the project demonstrates the effective application of DBMS concepts in a real-world scenario. Overall, MediCare aims to improve data organization, accessibility, and consistency while providing a foundation for future enhancements in security, automation, analytics, and scalability.

11.2 Future enhancements

• Add a patient portal for appointment booking and access to personal reports.

• Integrate online payments and insurance claim workflows.
• Add SMS/email notifications for appointments and important updates.
• Introduce audit logging, scheduled backups and a documented recovery process.
• Expand automated tests for concurrent bookings, inventory updates and authorization.
• Improve reporting and query performance as the volume of data grows.
• Consider secure cloud deployment and a mobile-friendly patient interface.

References

1. PostgreSQL Global Development Group, “PostgreSQL Documentation.” Available at: https://www.postgresql.org/docs/
2. Node.js, “Node.js Documentation.” Available at: https://nodejs.org/docs/

3.OpenAI, “ChatGPT.” Available at: https://chatgpt.com/

4. Anthropic, “Claude.” Available at: https://claude.ai/

5. GitHub, “Hospital Management System DBMS Project Repository.” Available at: https://github.com/mydhilyms/HOSPITAL-MANAGEMENT-SYSTEM-DBMS-Project-